

Report IN3030 – Oblig 3

Processor:

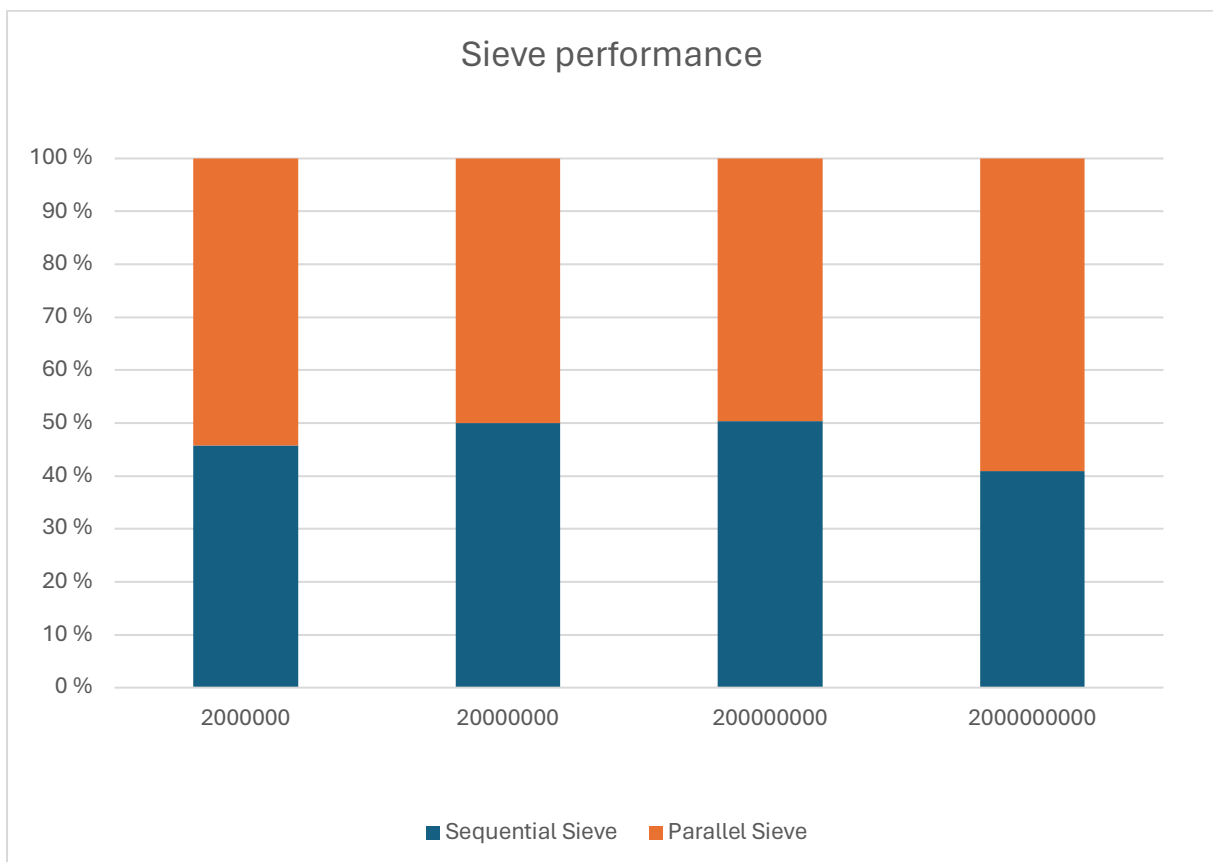
- Apple M1-chip, 3228 MHz
- 8 core cpu

Implementation:

When it came to the sequential sieve, I just used the given sieve of Eretosthenes from the precode.

The idea behind the paralyzed version of the sieve was to divide up N (given value) into section for each thread to work. The thread is split into two parts. First the thread marks its section, thereafter waiting for the rest of the threads to check their section. The last thread to finish, signals the rest of the threads to check if it is a prime and the thread-class ParaSieve will return the number of primes. Thereafter the monitor is called and returns the primes $\leq$ N. The return of primes is done sequentially.

n	Sequential Sieve	Parallel Sieve
2000000	6,68	7,91
20000000	64,23	64,14
200000000	642,65	634,57
2000000000	9001,09	12965,11

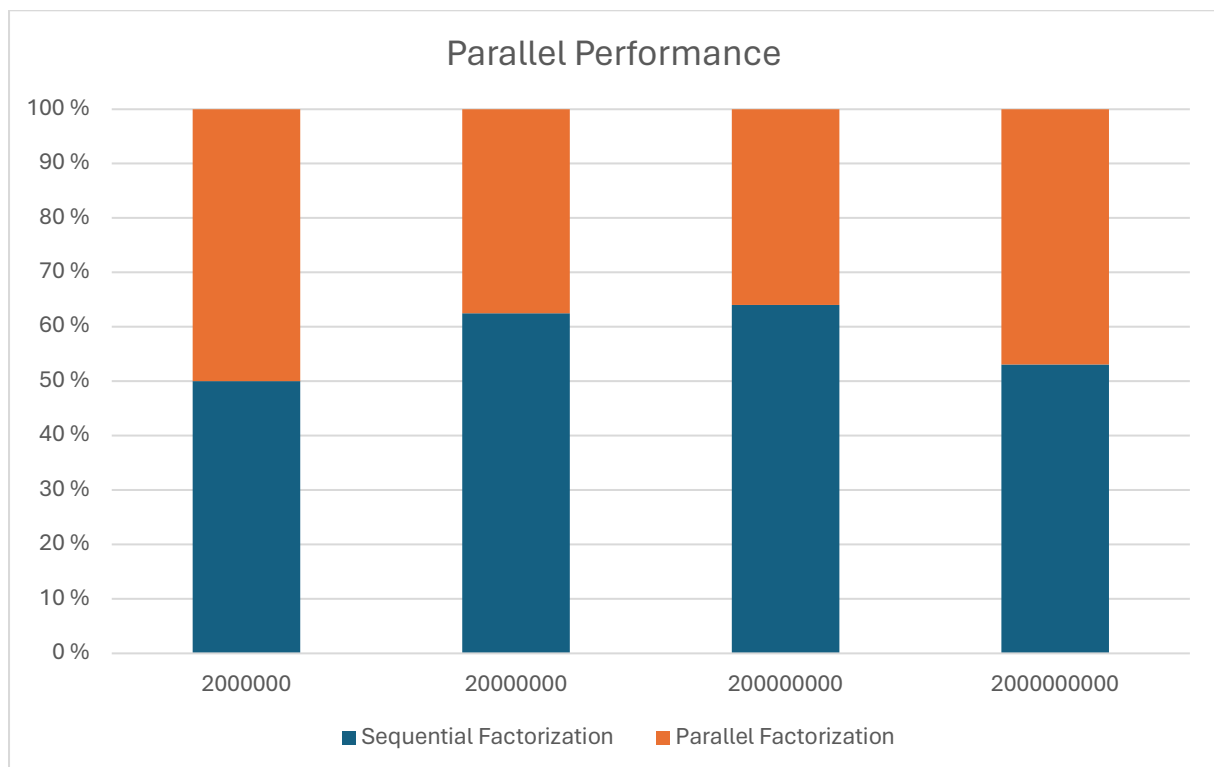


The implementation of the Sequential factorization starts by first initializing the sieve of eretosthenes. Then we multiply N with itself to get the biggest 100 numbers. We modulo all the value with the primes to see if it equals 0. If so, we know that it can be factorized with that prime. If it goes through the loop and it has not changed, then the number itself is a prime.

The implementation of the parallel version first starts with the parallelized version of the sieve. Each thread is given a section of the table of primes and checks the same number. After all the threads has been done checking, they put their results together. We check if it is correct by multiplyeing the result together and see if it equals the number factorized. If not, we add the missing value to the result.

If it is the last thread, it signals the rest of the threads. If not, the thread awaits.

n	Sequential Factorization	Parallel Factorization
2000000	19,92	19,91
20000000	165,83	99,44
200000000	1544,26	865,76
2000000000	16820,78	14881,21



Conclusion:

Para Sieve vs Seq Sieve:

The there are no significant speedups to be seen, and in the case of the 2.000.000.000 the parallized version did way worse than the sequential. This might come from the initialization of the initial primes combined with the last part getting all the primes on a single thread. This might cause some timeloss, or even big time loss.

Para Factorization vs Seq Factorization

We can see from the results that it is almost even when N is small, but for the middle ranges the parallelized speedups are more significant. There might be some different reasons for this, but I think that the generating of prime numbers might be the biggest factor as N grows. Since the para sieve is slow, we lose processing time. In some cases all the primes can be on one thread and the rest of the threads has to wait. If that's the case, the sequential version is actually faster.